P.Vishwa nandan(192473015)

TASK 5 – SAME SCREEN, DIFFERENT EVENT-HANDLING STYLES (7 Marks)

1. Aim

To implement a Java Swing currency converter using four different event-handling techniques: implementing ActionListener in the frame, a named inner class, an anonymous inner class, and a lambda expression.

The application must accept an amount in Indian Rupees (INR), allow the user to select USD or EUR, calculate the converted amount, and display the result in a label and dialog box.

2. Problem Description

A currency converter application requires the following components:

1. An amount text field to enter the INR value.
2. A currency selection ComboBox containing USD and EUR.
3. A Convert button to perform the conversion.
4. A result label to display the converted amount.

The application must also:

- Handle window closing using WindowAdapter.
- Convert the amount when the Convert button is clicked.
- Convert the amount when Enter is pressed in the amount field.
- Display the result in a dialog box.
- Display an error message when the entered amount is invalid.

The conversion rates used for demonstration are:

- 1 INR = 0.012 USD
- 1 INR = 0.011 EUR

These are fixed sample rates, not live exchange rates.

3. Task 5.1 – Four Implementations (3 Marks)

The same currency converter is implemented using four different event-handling techniques.

3.1 Implementation A – Frame Implements ActionListener

Explanation

In this approach, the main JFrame class implements the ActionListener interface.

The actionPerformed() method is written directly inside the frame class.

The frame itself handles button clicks and Enter-key events.

Working:

1. The frame implements ActionListener.
2. The Convert button registers the frame as its listener.
3. The amount text field also registers the frame as its listener.
4. When either event occurs, actionPerformed() executes.
5. The conversion method calculates the result and displays it.

Java Program – ConverterFrameListener.java

Explanation:

- implements ActionListener allows the frame to receive action events.
- addActionListener(this) registers the frame as the listener.
- actionPerformed() calls the conversion method.
- The conversion method validates the amount and calculates the result.
- WindowAdapter handles the closing event.

3.2 Implementation B – Named Inner Class

Explanation

In this approach, a separate named inner class implements ActionListener.

The inner class handles the button click and Enter-key events.

It can directly access the fields and methods of the outer JFrame.

Working:

1. Create a named inner class called ConvertHandler.
2. Implement ActionListener inside it.
3. Create an object of the inner class.
4. Register that object with the Convert button and amount field.
5. Call the conversion method when an event occurs.

Java Program – ConverterNamedInner.java

Explanation:

- ConvertHandler is a named inner class.
- It implements ActionListener.

- Its actionPerformed() method calls convert().
- The same handler object is registered with both the button and text field.
- This keeps event-handling code separate from the main GUI construction.

3.3 Implementation C – Anonymous Inner Class

Explanation

An anonymous inner class is a class without an explicit name.

It is created directly when registering the event listener.

Instead of creating a separate named handler class, the ActionListener implementation is written inside the constructor.

Working:

1. Create an ActionListener using an anonymous inner class.
2. Override the actionPerformed() method.
3. Call the conversion method inside it.
4. Register the same listener object with the Convert button and amount field.

Java Program – ConverterAnonymous.java

Explanation:

- The ActionListener is created without a class name.
- actionPerformed() handles the event.
- The same listener object is registered with both components.
- This avoids creating a separate named inner class.
- It is useful when an event handler is needed for a small operation.

3.4 Implementation D – Lambda Expression

Explanation

A lambda expression provides a shorter way to implement a functional interface such as ActionListener.

Instead of writing an entire anonymous inner class, the event-handling operation can be written in one line.

Working:

1. Create an ActionListener using a lambda expression.
2. Call the conversion method inside the lambda.

3. Register the same listener with the Convert button and amount field.
4. Display the conversion result.

Java Program – Converter.java

Explanation:

- e -> convert() is the lambda expression.
- The same lambda listener is assigned to both the button and text field.
- Clicking Convert triggers the conversion.
- Pressing Enter in the amount field also triggers the conversion.
- The result is displayed using JLabel and JOptionPane.
- Invalid input displays the message Invalid amount.

4. Task 5.2 – Change the Requirement (2 Marks)

4.1 New Requirements

The original converter application only performs conversion when the Convert button is clicked.

The new requirements are:

1. The application must also convert when Enter is pressed in the amount text field.
2. The converted result must be displayed in a dialog box.
3. Invalid input must display an error dialog.

4.2 Changes Made in Each Implementation

A. Frame Implements ActionListener

Original registration:

convertButton.addActionListener(this);

Modified registration:

convertButton.addActionListener(this);

amount.addActionListener(this);

The frame now receives events from both components.

The existing actionPerformed() method handles both events.

The conversion method is also updated to display the result using:

```
JOptionPane.showMessageDialog(
    this,
    message,
    "Conversion Result",
    JOptionPane.INFORMATION_MESSAGE
);
```

Code changed: One additional listener-registration statement and the result-dialog code.

B. Named Inner Class

Original registration:

```
convertButton.addActionListener(handler);
```

Modified registration:

```
convertButton.addActionListener(handler);

amount.addActionListener(handler);
```

The same named inner class handles both events.

The conversion method is updated to display the result dialog.

Code changed: One additional listener-registration statement and the result-dialog code.

C. Anonymous Inner Class

Original registration:

```
convertButton.addActionListener(handler);
```

Modified registration:

```
convertButton.addActionListener(handler);

amount.addActionListener(handler);
```

The same anonymous listener object is reused for both components.

The conversion method is modified to show the dialog.

Code changed: One additional listener-registration statement and the result-dialog code.

D. Lambda Expression

Original registration:

convertButton.addActionListener(e -> convert());

Modified registration:

ActionListener convertAction = e -> convert();

convertButton.addActionListener(convertAction);

amount.addActionListener(convertAction);

The same lambda listener is assigned to both components.

The conversion method is updated to display the result dialog.

Code changed: The lambda is stored in a variable, registered with two components, and the result-dialog code is added.

4.3 Comparison of Code Changes

Implementation	Enter-Key Support	Dialog Box	Modification Required
Frame implements ActionListener	Register frame with text field	Modify convert()	Small
Named inner class	Register handler with text field	Modify convert()	Small
Anonymous inner class	Register shared handler with text field	Modify convert()	Small
Lambda expression	Register shared lambda with text field	Modify convert()	Very small

All four approaches can handle the new requirement. Reusing the same listener avoids writing duplicate conversion logic.

5. Task 5.3 – Compare the Styles (2 Marks)

5.1 Comparison of Four Event-Handling Techniques

Feature	Frame Implements	Named Inner Class	Anonymous Inner Class	Lambda
Code length	Moderate	Moderate	Longer	Short

Readability	Good for small programs	Good	Moderate	Very good for simple actions
Access to frame fields	Direct	Direct	Direct	Direct
Reusability	Frame acts as listener	Handler can be reused	Reusable if stored in a variable	Reusable if stored in a variable
Maintainability	Can make frame crowded	Separates event logic	Handler is embedded in constructor	Simple for small operations
Best use	Small GUI programs	Larger or reusable handlers	One-time custom handlers	Simple event operations

5.2 Advantages and Disadvantages

1. Frame Implements ActionListener

Advantages:

- Easy to understand.
- Direct access to frame fields.
- Suitable for small applications.

Disadvantages:

- The frame contains both GUI and event-handling logic.
- Large applications can become difficult to maintain.

2. Named Inner Class

Advantages:

- Separates event-handling logic from GUI construction.
- Easy to understand and debug.
- Can be reused for multiple components.

Disadvantages:

- Requires additional class code.
- Slightly longer than a lambda expression.

3. Anonymous Inner Class

Advantages:

- Does not require a separately named class.
- Can directly access outer class fields.
- Useful for one-time event handlers.

Disadvantages:

- Makes the constructor longer.
- Can reduce readability when the handler contains large amounts of code.

4. Lambda Expression

Advantages:

- Short and readable.
- Reduces unnecessary code.
- Suitable for simple event-handling operations.
- Easy to reuse by storing it in a variable.

Disadvantages:

- Not ideal for large event handlers.
- Complex logic inside a lambda may be difficult to understand.

5.3 Which Style Is Best for a Large Application?

For a large application, a named inner class or a reusable Swing Action is preferable.

A named inner class keeps event-handling logic separate and makes debugging easier.

A Swing Action is particularly useful when the same operation must be triggered through multiple components, such as a button, menu item, and keyboard shortcut.

For the simple currency converter, the lambda expression is the most convenient approach because it requires less code and is easy to understand.

6. Alternative Concept and Complexity Analysis

6.1 Alternative 1 – Method Reference

Instead of writing a lambda expression, a method reference can be used to refer to an existing method.

Example:

```
private void handleConvert(ActionEvent e) {
```

```
    convert();
}
```

Register the method reference:

```
ActionListener handler = this::handleConvert;


convertButton.addActionListener(handler);


amount.addActionListener(handler);
```

Explanation:

- this::handleConvert refers to the existing method.
- The method receives an ActionEvent.
- The same handler can be registered with multiple components.
- It avoids writing the lambda body separately.

A no-argument method such as convert() cannot be directly assigned to ActionListener because ActionListener's method receives an ActionEvent parameter. The handleConvert(ActionEvent e) method matches the required signature.

6.2 Alternative 2 – Shared Swing Action

A Swing Action is a reusable object that represents an operation.

It can be connected to:

- A JButton.
- A JMenuItem.
- A keyboard shortcut.

Example:

```
import java.awt.event.*;
import javax.swing.*;


public class SharedActionExample {


    public static void main(String[] args) {
```

```
JFrame frame = new JFrame("Shared Action Example");

JTextField amount = new JTextField(10);

JLabel result = new JLabel(" ");

JButton convertButton;

// Create one shared Action object
Action convertAction = new AbstractAction("Convert") {

    @Override
    public void actionPerformed(ActionEvent e) {

        result.setText("Conversion requested.");
    }
};

convertButton = new JButton(convertAction);

// Create menu item using the same Action
JMenuItem menuItem = new JMenuItem(convertAction);

JMenu menu = new JMenu("Operations");

menu.add(menuItem);

JMenuBar menuBar = new JMenuBar();
```

```
menuBar.add(menu);

frame.setJMenuBar(menuBar);

// Add components
frame.setLayout(new java.awt.FlowLayout());

frame.add(amount);

frame.add(convertButton);

frame.add(result);

// Enter key listener
amount.addActionListener(convertAction);

// Ctrl + Enter keyboard shortcut
frame.getRootPane().getInputMap(
    JComponent.WHEN_IN_FOCUSED_WINDOW
).put(
    KeyStroke.getKeyStroke("ctrl ENTER"),
    "convert"
);

frame.getRootPane().getActionMap().put(
    "convert",
    convertAction
);
```

```
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        frame.pack();

        frame.setLocationRelativeTo(null);

        frame.setVisible(true);
    }
}
```

Explanation:

One Swing Action object is created to represent the Convert operation.

The same Action object is connected to the Convert button, menu item, amount field, and keyboard shortcut.

Therefore, the same operation can be triggered from different parts of the application without implementing separate conversion logic.

6.3 Comparison of Alternative Approaches

Feature	Lambda	Method Reference	Swing Action
Code length	Short	Short	More initial code
Reuse	Yes, if stored	Yes, if stored	Yes
Button support	Yes	Yes	Yes
Menu support	Requires registration	Requires registration	Built-in
Keyboard shortcut	Requires additional mapping	Requires additional mapping	Supports ActionMap/InputMap
Best use	Simple actions	Existing handler methods	Large Swing applications

6.4 Time Complexity Analysis

Let k represent the number of event sources that can trigger the Convert operation.

Frame Implements ActionListener:

The same actionPerformed() method handles the events.

Event dispatch:

O(1)

Named Inner Class:

The event is passed directly to the registered handler.

Event dispatch:

O(1)

Anonymous Inner Class:

The registered listener receives the event directly.

Event dispatch:

O(1)

Lambda Expression:

The lambda listener is invoked directly when the registered event occurs.

Event dispatch:

O(1)

Swing Action:

The same Action object handles the operation for different event sources.

Event dispatch:

O(1)

These complexities describe the listener dispatch step, not the time required for the currency calculation or any external operation.

6.5 Space Complexity Analysis

For a fixed number of event sources, each implementation can reuse a single handler object.

Implementation	**Additional Handler Objects**
Frame implements ActionListener	0 separate handler objects; the frame itself is the listener
Named inner class	1 handler object
Anonymous inner class	1 shared handler object
Lambda expression	1 shared lambda reference/object for the listener
Method reference	1 shared listener reference/object

Swing Action	1 shared Action object

If a separate listener is created for each of the k event sources, additional listener storage becomes:

O(k)

If one shared listener is reused, handler storage remains:

O(1)

The exact memory occupied by each listener depends on the Java implementation and captured references.

6.6 Engineering Decision

For a small application, lambda expressions are suitable because they provide shorter and clearer event-handling code.

For a large Swing application, a shared Swing Action is a better choice when the same operation is triggered by buttons, menus, and keyboard shortcuts.

It reduces duplicated event logic and makes it easier to enable, disable, or modify the operation in one place.

7. Testing and Verification

The following test cases verify the currency converter.

Test Case	Input / Action	Expected Result
TC1	Enter 1000 and select USD	12.00 USD
TC2	Enter 1000 and select EUR	11.00 EUR
TC3	Enter 500 and select USD	6.00 USD
TC4	Enter 500 and select EUR	5.50 EUR
TC5	Enter invalid text	Invalid amount
TC6	Leave amount field empty	Invalid amount
TC7	Enter a negative value	Invalid amount
TC8	Click Convert	Conversion occurs
TC9	Press Enter in amount field	Conversion occurs
TC10	Perform valid conversion	Result label and dialog appear

TC11	Perform invalid conversion	Error dialog appears
TC12	Click window close button	Application closes

7.1 Expected Output – USD Conversion

Input:

Amount: 1000

Target Currency: USD

Expected result:

1000.00 INR = 12.00 USD

7.2 Expected Output – EUR Conversion

Input:

Amount: 1000

Target Currency: EUR

Expected result:

1000.00 INR = 11.00 EUR

7.3 Expected Output – Invalid Amount

Input:

Amount: ABC

Target Currency: USD

Expected result:

Invalid amount

The application displays this message in the result label and an error dialog box.

8. Final Result

The currency converter application was implemented using four different Java event-handling techniques: Frame implements ActionListener, named inner class, anonymous inner class, and lambda expression.

The application supports conversion from INR to USD and EUR using sample exchange rates.

The new requirements were implemented by registering the handler with both the Convert button and amount text field. The result is displayed in a label and dialog box.

The four approaches were compared based on readability, reusability, field access, and maintainability. Method references and shared Swing Actions were also analysed as alternative approaches.

Thus, the application demonstrates different event-handling techniques in Java Swing.